

Inter-Service Communication Writeup

Leave Management System - Microservices

Document Version: 1.0

Last Updated: 02 June 2026

Table of Contents

1. [Executive Summary](#)
2. [Architecture Overview](#)
3. [Communication Patterns](#)
4. [Service Dependencies](#)
5. [Data Flow Diagrams](#)
6. [Communication Details](#)
7. [Assumptions](#)
8. [Error Handling & Resilience](#)
9. [Monitoring & Observability](#)
10. [Best Practices](#)

Executive Summary

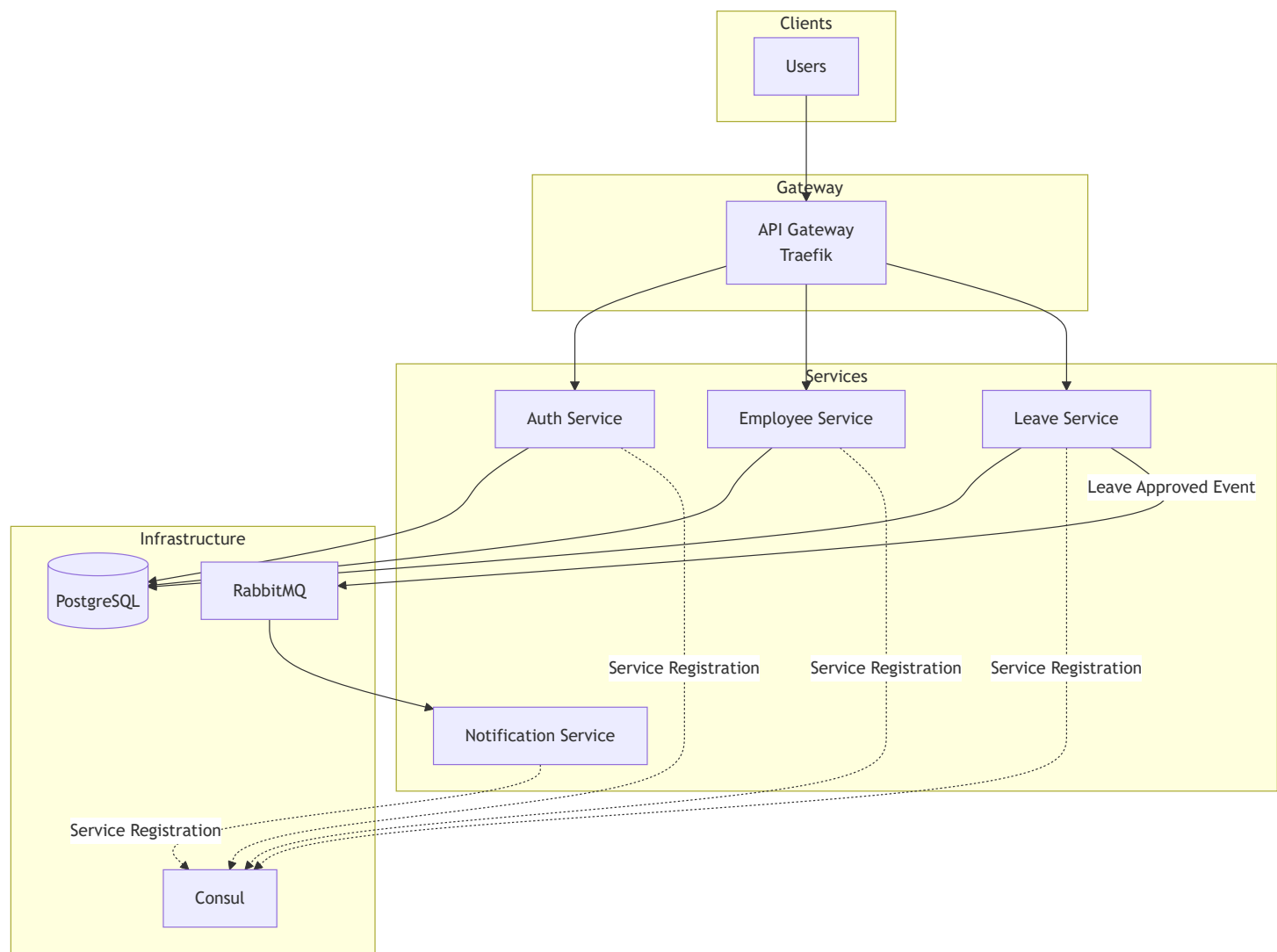
The Leave Management System is a distributed microservices architecture consisting of four primary services that communicate through multiple channels:

- **Synchronous HTTP/REST** for service-to-service RPC calls (with token propagation)
- **Asynchronous Message Broker (RabbitMQ)** for event publishing and notification distribution
- **Service Registry (Consul)** for service discovery and health checks

This document outlines the inter-service communication design, assumptions, and operational guidelines.

Architecture Overview

Service Topology



Components

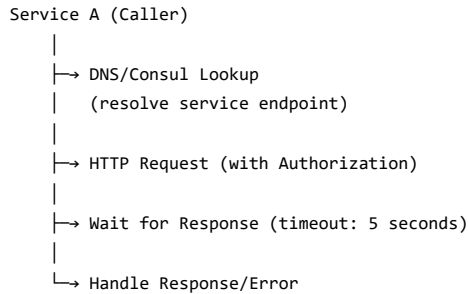
Component	Host	Port	Purpose
Traefik Gateway	traefik	80, 8080	Reverse proxy, request routing
Auth Service	auth-service	8001	JWT token generation & validation
Employee Service	employee-service	8002	Employee data & leave balances
Leave Service	leave-service	8003	Leave request management
Notification Service	notification-service	8004	Event consumption & notifications
Consul	consul	8500	Service discovery & health checks
PostgreSQL	postgres	5432	Persistent data storage
RabbitMQ	rabbitmq	5672, 15672	Message broker for async communication
OpenTelemetry Collector	otel-collector	4317 (gRPC), 4318 (HTTP)	Telemetry data collection
Jaeger	jaeger	16686	Distributed tracing UI

Communication Patterns

1. Synchronous Communication (HTTP/REST)

Used for: Direct service-to-service calls requiring immediate response

Pattern Details



Characteristics:

- Request/Response model
- Blocking operation
- Timeout protection (5 seconds)
- Error propagation
- Authorization header forwarding

Current Implementations:

1. Leave Service → Employee Service (Check leave balance)

```
URL: http://employee-service:8002/employees/{username}/leave-balance/{leave_type}
Method: GET
Headers: Authorization: Bearer {jwt_token}
Response: { "remaining": 12, "used": 3, "allocated": 15 }
```

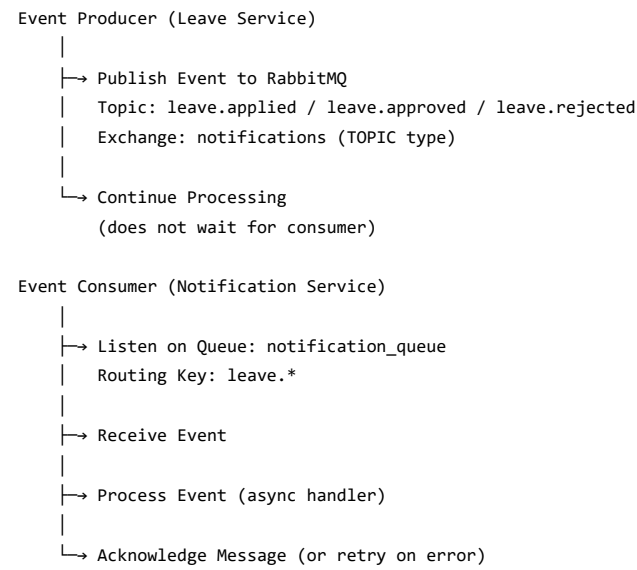
2. Leave Service → Employee Service (Deduct balance on approval)

```
URL: http://employee-service:8002/employees/{username}/deduct-balance
Method: PUT
Headers: Authorization: Bearer {jwt_token}
Payload: { "leave_type": "CASUAL", "days": 5 }
Response: { "success": true, "new_balance": 7 }
```

2. Asynchronous Communication (RabbitMQ)

Used for: Event publishing and notification distribution (fire-and-forget)

Pattern Details



Characteristics:

- Publisher doesn't wait for response
- Durable queues and messages
- TOPIC exchange with routing keys
- Multiple consumers possible
- Asynchronous processing

Event Types:

Event Name	Publisher	Routing Key	Payload	Trigger
leave.applied	Leave Service	leave.applied	{ employee_username, leave_type, start_date, end_date, reason }	When employee applies for leave
leave.approved	Leave Service	leave.approved	{ employee_username, leave_id, approver_username, start_date, end_date, leave_type }	When manager approves leave request
leave.rejected	Leave Service	leave.rejected	{ employee_username, leave_id, rejector_username, reason, manager_comment }	When manager rejects leave request

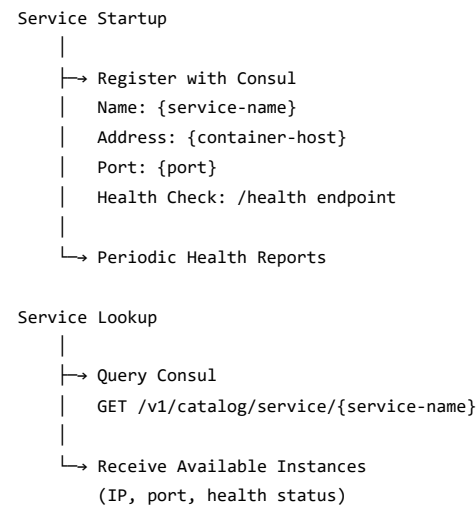
Queue Configuration:

Exchange: notifications (Type: TOPIC, Durable: yes)
Queue: notification_queue (Durable: yes, Auto-delete: no)
Binding: notification_queue -> notifications (routing key: leave.*)
Prefetch: 1 (one message at a time per consumer)

3. Service Discovery (Consul)

Used for: Dynamic service location and health management

Pattern Details



Registration Example:

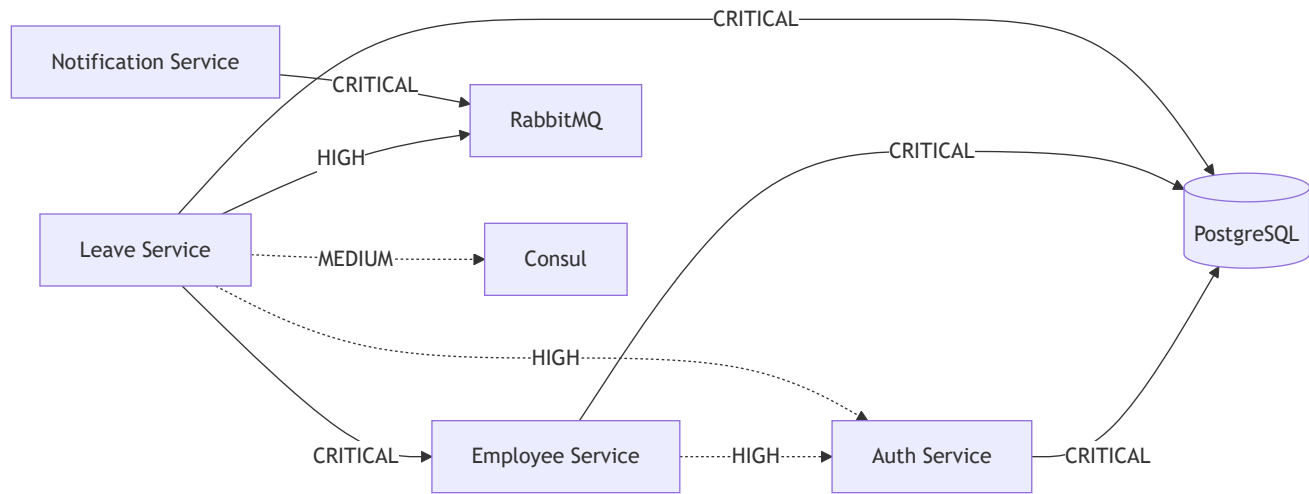
```
consul_client.agent.service.register(
    name="leave-service",
    service_id="leave-service-1",
    address="leave-service",
    port=8003
)
```

Health Checks:

- **Type:** HTTP
- **Endpoint:** /health
- **Interval:** 10 seconds
- **Timeout:** 5 seconds
- **Deregister on Failure:** 30s

Service Dependencies

Dependency Graph



Service Dependency Matrix

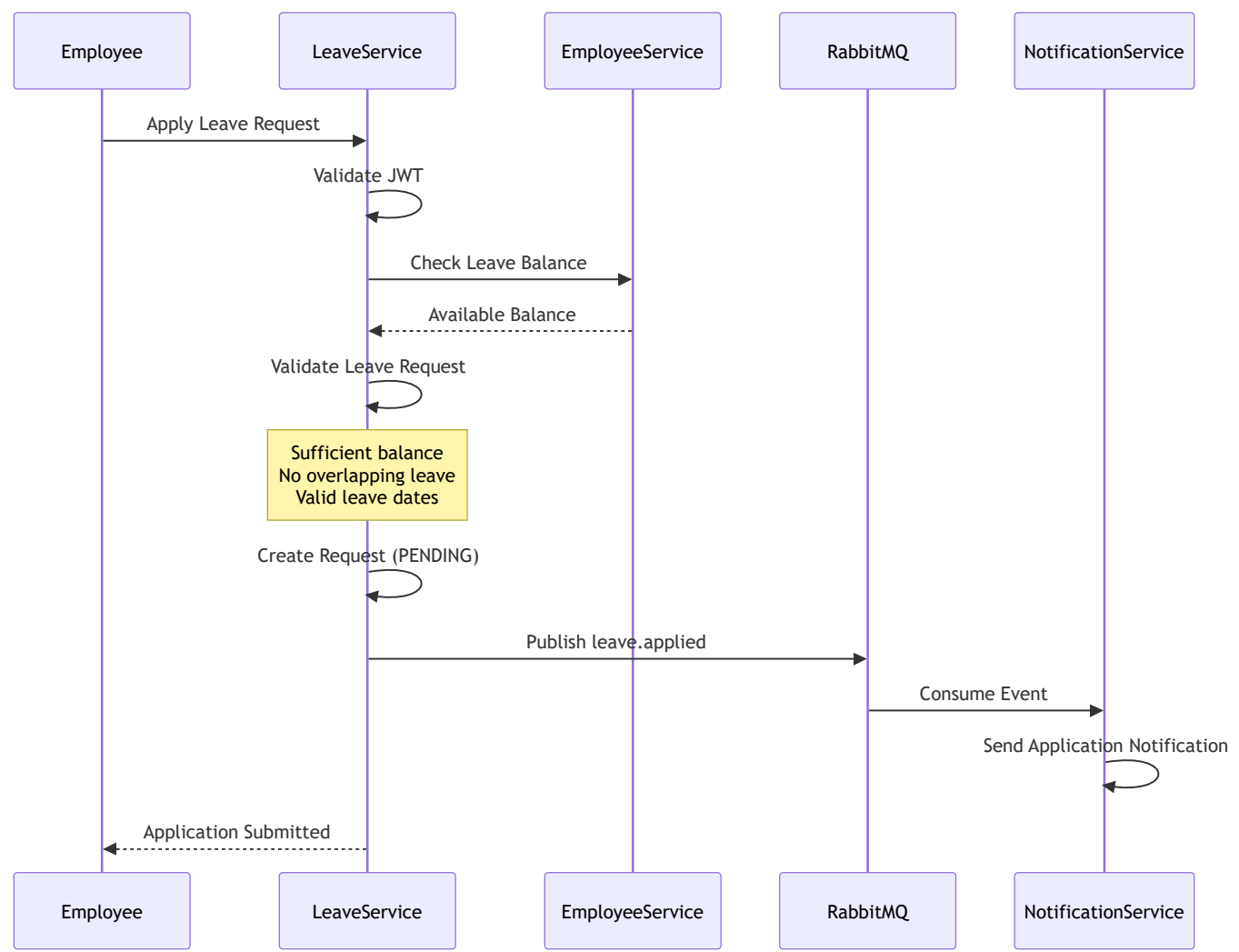
Service	Depends On	Communication Type	Criticality
Auth Service	PostgreSQL	Database	CRITICAL
Employee Service	PostgreSQL	Database	CRITICAL
	Auth Service (validation only)	Middleware	HIGH
Leave Service	PostgreSQL	Database	CRITICAL
	Auth Service (validation only)	Middleware	HIGH
	Employee Service	HTTP/REST	CRITICAL
	RabbitMQ	AMQP/Message Queue	HIGH
	Consul	HTTP	MEDIUM
Notification Service	RabbitMQ	AMQP/Message Queue	CRITICAL

Criticality Levels:

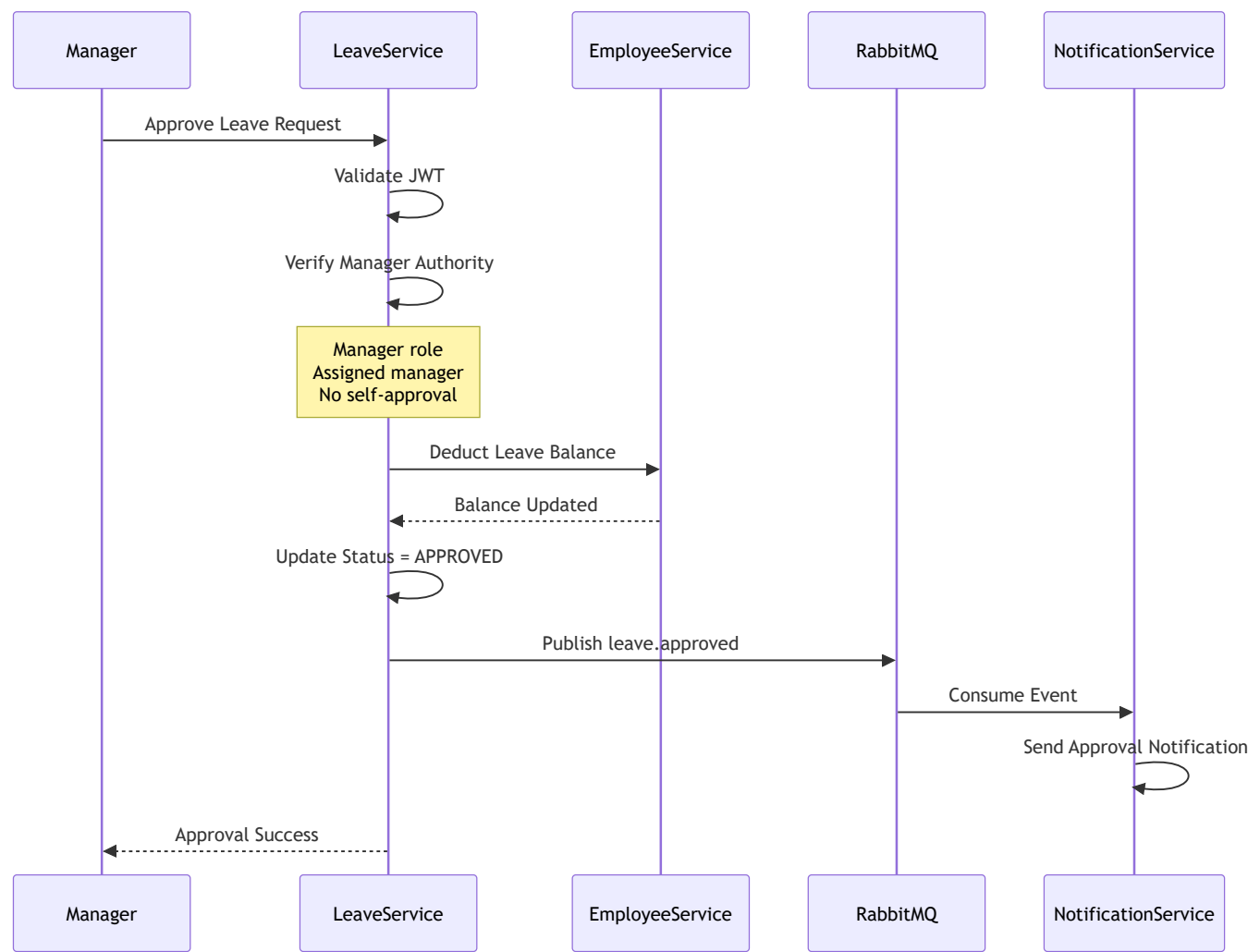
- **CRITICAL:** Service fails if dependency fails; blocking operation
- **HIGH:** Service functionality degraded; async operation
- **MEDIUM:** Service features limited; non-blocking operation

Data Flow Diagrams

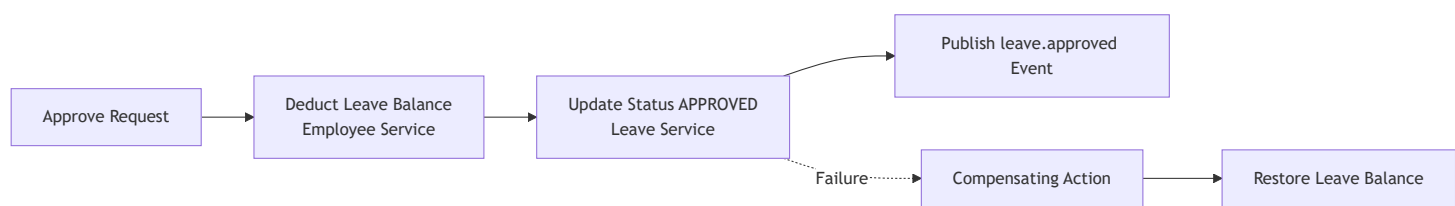
1. Leave Application Flow



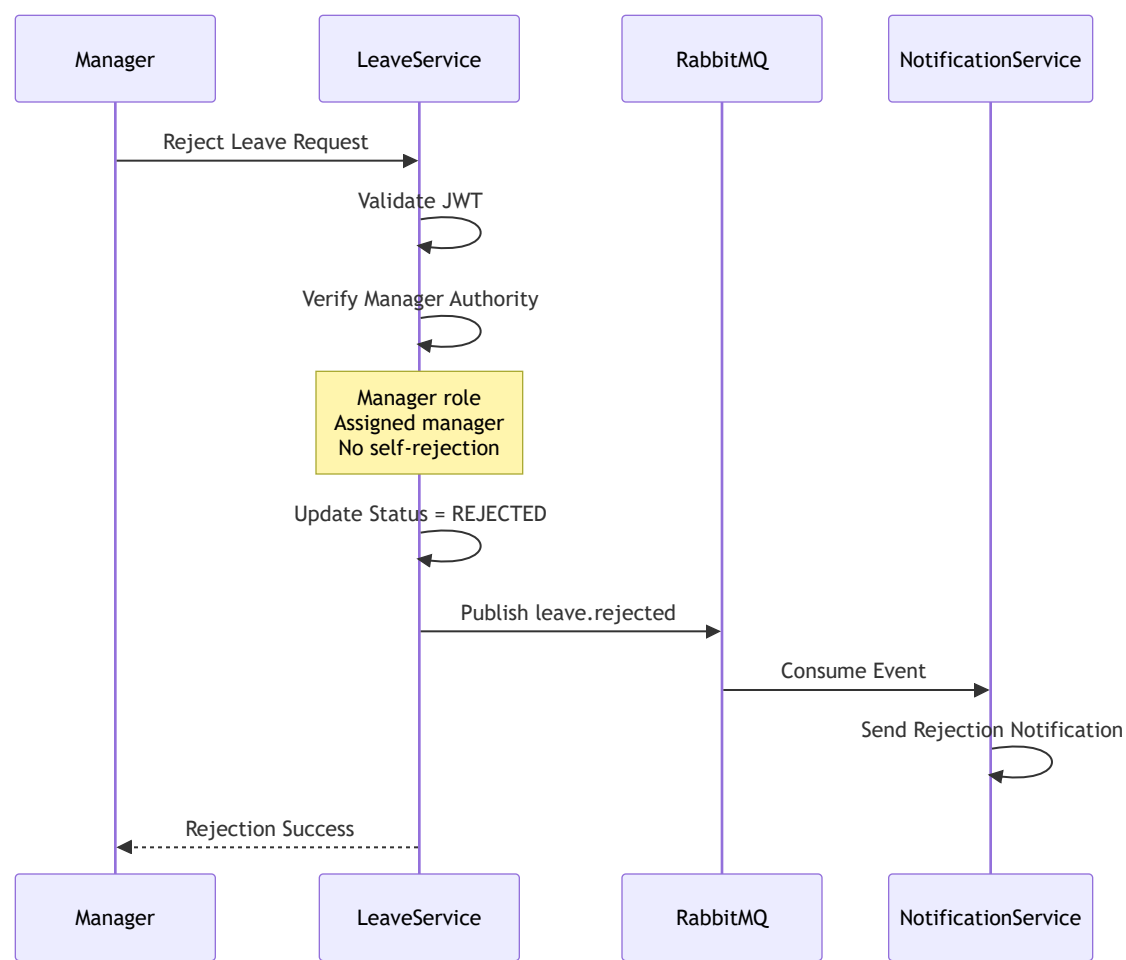
2. Leave Approval Flow (with Balance Deduction)



3. Leave Approval Saga (Compensation Flow)



4. Leave Rejection Flow



Communication Details

HTTP/REST Calls

1. Check Leave Balance

Caller: Leave Service
Callee: Employee Service
Trigger: User applies for leave

```
GET /employees/{username}/leave-balance/{leave_type}
```

```
Authorization: Bearer {jwt_token}
```

```
Host: employee-service:8002
```

```
# Response (200)
```

```
{
  "leave_type": "CASUAL",
  "allocated": 15,
  "used": 3,
  "remaining": 12
}
```

```
# Response (400)
```

```
{
  "detail": "Employee or leave type not found"
}
```

```
# Response (503)
```

```
{
  "detail": "Employee Service temporarily unavailable"
}
```

Error Scenarios:

- **404 (Employee Not Found):** Return 400 to client
- **500 (Employee Service Error):** Return 503 to client
- **Timeout (>5s):** Return 503 to client

2. Deduct Leave Balance

Caller: Leave Service

Callee: Employee Service

Trigger: Manager approves leave request

```
PUT /employees/{username}/deduct-balance
```

```
Authorization: Bearer {jwt_token}
```

```
Content-Type: application/json
```

```
Host: employee-service:8002
```

```
{
  "leave_type": "CASUAL",
  "days": 5
}
```

```
# Response (200)
```

```
{
  "success": true,
  "new_balance": 7,
  "leave_type": "CASUAL"
}
```

```
# Response (400)
```

```
{
  "detail": "Insufficient leave balance"
}
```

```
# Response (404)
```

```
{
  "detail": "Employee not found"
}
```

RabbitMQ Message Publishing

Event Message Format

```
{
  "header": {
    "event_type": "leave.applied|leave.approved|leave.rejected",
    "timestamp": "2026-06-03T14:30:00Z",
    "trace_id": "{correlation-id}",
    "source_service": "leave-service"
  },
  "payload": {
    "leave_id": 123,
    "employee_username": "emp001",
    "leave_type": "CASUAL",
    "start_date": "2026-06-10",
    "end_date": "2026-06-14",
    "number_of_days": 5,
    "reason": "Personal reasons",
    "status": "PENDING|APPROVED|REJECTED",
    "manager_username": "mgr001",
    "manager_comment": "Approved",
    "approved_at": "2026-06-03T14:30:00Z",
    "rejected_at": null
  }
}
```

Publishing Code Pattern

```
async def publish_notification(event_type: str, payload: dict):
    """
    Args:
        event_type: Routing key (leave.applied, leave.approved, etc.)
        payload: Event data dictionary
    """
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    async with connection:
        channel = await connection.channel()
        exchange = await channel.declare_exchange(
            "notifications",
            ExchangeType.TOPIC,
            durable=True
        )
        message = Message(
            body=json.dumps(payload).encode(),
            delivery_mode=DeliveryMode.PERSISTENT
        )
        await exchange.publish(message, routing_key=event_type)
```

Consul Service Registration

Auto-Discovery on Startup

```
consul_client.agent.service.register(  
    name="leave-service",  
    service_id="leave-service-1",  
    address=SERVICE_HOST, # Container hostname  
    port=SERVICE_PORT     # Service port  
)  
  
# Consul Health Check (automatic)  
# GET http://leave-service:8003/health  
# Expected: 200 OK
```

Service Lookup (Client-side)

```
services = consul_client.catalog.service("employee-service")  
# Returns: [(host, port), ...]  
# Use for dynamic endpoint resolution
```

Assumptions

Infrastructure Assumptions

- 1. Network Connectivity**
 - All services can communicate via Docker network
 - Service discovery via DNS (within docker-compose network)
 - No network partitions expected in single-node deployment
- 2. Docker Container Environment**
 - Services run as Docker containers (specified in docker-compose.yml)
 - Container-to-container communication via service names (e.g., "employee-service")
 - Environment variables injected at runtime
- 3. Shared Database**
 - Single PostgreSQL 16 instance shared by all services
 - Database: leave_management
 - Connection pool: 20 connections
 - Assumes DB is always available (critical infrastructure)
- 4. Message Broker Availability**
 - RabbitMQ runs continuously
 - Messages are persisted (durable queues/exchanges)
 - Single node RabbitMQ (no clustering)
 - No message expiration configured

Service Behavior Assumptions

- 5. JWT Token Validity**
 - Tokens are valid for 24 hours
 - Secret key: cryptographically secure (should use environment variable)
 - All protected endpoints require Bearer token in Authorization header
 - Token validation happens at middleware level (middleware-first pattern)
- 6. Authentication Flow**
 - User authenticates with Auth Service first
 - Receives JWT token
 - Token passed to other services via Authorization header
 - Services validate token using same secret key

- **Assumption: All services use same SECRET_KEY**

7. Employee-Service Relationship

- Each user has an associated Employee record
- Username is unique identifier across services
- Manager assignment is hierarchical (manager_id foreign key to self)
- Leave balances are pre-initialized on employee creation

8. Leave Request Processing

- Leave requests require manager approval before balance deduction
- Balance deduction is atomic (transactional)
- Once approved, leave cannot be modified or cancelled
- Overlapping leave requests on same dates are rejected

Communication Pattern Assumptions

9. HTTP Call Timeouts

- All inter-service HTTP calls timeout at 5 seconds
- If timeout occurs, service returns 503 to client
- No automatic retry at HTTP layer

10. RabbitMQ Message Delivery

- Messages are guaranteed delivery (persistent/durable)
- Once published, message will be delivered to all subscribers
- If Notification Service is down, messages queue up
- **Assumption: Notification Service will eventually consume messages**

11. Service Discovery

- Consul is the source of truth for service locations
- Health checks run every 10 seconds
- Failed service automatically deregistered after 30 seconds
- **Assumption: Consul runs continuously**

Data Assumptions

12. Timezone Handling

- All dates stored in UTC in database
- `date.today()` operates on server's system timezone
- **Assumption: Server runs in UTC or consistent timezone**

13. Leave Type Validity

- Valid leave types: CASUAL, SICK, EARNED, UNPAID
- Leave types are immutable (defined in shared enums)
- Employee must have balance record for all 4 leave types

14. User Roles

- Valid roles: EMPLOYEE, MANAGER
- Manager can only approve/reject leave for team members
- **Assumption: Org hierarchy is maintained accurately**

Error Handling Assumptions

15. Graceful Degradation

- If Employee Service is unavailable, Leave Service cannot process leave applications
- Leave Service returns 503 to client instead of crashing
- Notification failures do not block leave operations (async)

16. Database Transaction Handling

- Leave approval uses transaction (all-or-nothing)
- On transaction failure, leave status remains PENDING
- **Assumption: No concurrent updates on same leave request**

17. Duplicate Request Handling

- Constraint on (employee_username, start_date, end_date) prevents duplicates
- If duplicate detected, returns 409 Conflict
- **Assumption: Unique constraint is enforced at DB level**

Observability Assumptions

18. Distributed Tracing
 - OpenTelemetry collector running at `otel-collector:4317`
 - All services instrument HTTP clients and DB queries
 - Jaeger UI available at `http://jaeger:16686`
 - **Assumption: Trace context is propagated across service calls**
19. Logging
 - All services log to stdout
 - Docker collects logs (available via `docker logs`)
 - Log level: INFO (debug logs in development)
 - **Assumption: Centralized logging not configured**

Error Handling & Resilience

HTTP Error Handling

Leave Service → Employee Service Error Scenarios

Scenario	HTTP Status	Action	Client Response
Employee not found	404	Log error, treat as invalid employee	400 Bad Request
Insufficient balance	400	Log validation failure	400 Bad Request
Employee Service timeout	(timeout)	Log timeout, abort operation	503 Service Unavailable
Employee Service unreachable	(connection error)	Log error, abort operation	503 Service Unavailable
Employee Service 500 error	500	Log error, abort operation	503 Service Unavailable
Authorization header missing	401	Log security event	401 Unauthorized

Error Code Mapping

- Client Error (4xx):
- └─ 400 Bad Request
 - └─ Invalid input, insufficient balance, employee not found
 - └─ 401 Unauthorized
 - └─ Missing or invalid JWT token
 - └─ 403 Forbidden
 - └─ User not authorized (e.g., manager approving own request)
 - └─ 409 Conflict
 - └─ Overlapping leave request, duplicate constraint violation
- Server Error (5xx):
- └─ 500 Internal Server Error
 - └─ DB error, unexpected exception
 - └─ 503 Service Unavailable
 - └─ Dependency service down, timeout, connection refused

Resilience Patterns Implemented

1. Timeout Protection

```
async with http.AsyncClient(timeout=5.0) as client:
    response = await client.get(url, timeout=5.0)
```
2. Circuit Breaker-Ready Architecture
 - All external calls wrapped in try-catch

- Proper error codes returned for monitoring
- **Not yet implemented:** Circuit breaker pattern

3. Graceful Degradation

- If Employee Service unavailable: Leave Service returns 503
- Client-side retry responsibility
- Notification failures don't block leave operations

4. Data Consistency

- Transactions for approval + balance deduction
- Constraints prevent invalid states
- **Assumption: No distributed transactions needed**

Failure Scenarios & Recovery

Scenario 1: Employee Service Unavailable

```

User tries to apply for leave
↓
Leave Service calls Employee Service (HTTP timeout after 5s)
↓
Catch httpx.TimeoutException
↓
Return 503 Service Unavailable to client
↓
Client implements retry logic (with backoff)
↓
Employee Service recovers
↓
Retry succeeds

```

Recovery Time: Until Employee Service is healthy + client retries

Scenario 2: RabbitMQ Broker Down

```

Manager approves leave (approval succeeds)
↓
Leave Service publishes leave.approved event
↓
RabbitMQ connection fails
↓
Log exception, but don't fail the approval response
↓
Client sees approval success (200 OK)
↓
RabbitMQ recovers
↓
Messages from queue are delivered
↓
Notification Service processes notifications

```

Impact: Messages delayed, but no data loss (persistent queue)

Scenario 3: Database Connection Failure

```
User tries to apply for leave
↓
Leave Service queries DB for overlaps
↓
PostgreSQL connection fails (too many connections or unavailable)
↓
SQLAlchemy raises exception
↓
Caught in try-except
↓
Return 500 Internal Server Error
↓
Client receives error
↓
DB recovers
↓
Retry succeeds
```

Impact: Operations blocked until DB recovers

Monitoring & Observability

Distributed Tracing

Setup:

- OpenTelemetry Collector: otel-collector:4317 (gRPC endpoint)
- Jaeger UI: <http://jaeger:16686>
- Auto-instrumentation: HTTPX, SQLAlchemy, FastAPI

Trace Context Propagation:

```
Client Request
├─ Header: traceparent (W3C format)
├─ Header: tracestate
└─
└─ Traced through all services:
    Leave Service
    │   HTTP call to Employee Service (traced)
    │   DB query (SQLAlchemy traced)
    └─ RabbitMQ publish (traced)
```

Trace IDs in Logs:

```
logger.info(f"Leave approved - trace_id: {trace_id}")
# Jaeger UI: Click trace_id to see full request flow
```

Health Check Endpoints

All services implement:

```
GET /health
Response: 200 OK with status details
```

Used by:

- Consul health checks (interval: 10s)

- Kubernetes liveness probes (if migrated)
- Load balancer health verification

Logging Strategy

Log Levels:

- **INFO:** Important flow events (approval, rejection, publication)
- **WARNING:** Retryable errors, timeouts
- **ERROR:** Non-retryable errors, service failures
- **DEBUG:** Variable values, step-by-step execution

Log Tags per Service:

Service	Log Tag	Example
Auth Service	auth	[auth] User login successful: emp001
Employee Service	employee	[employee] Leave balance fetched: CASUAL=12
Leave Service	leave	[leave] Leave approved: leave_id=123
Notification Service	notification	[notification] Event received: leave.approved

Best Practices

1. Authorization Header Forwarding

Pattern:

```
# Get authorization header from client request
auth_header = request.headers.get("authorization")

# Forward to downstream service
headers = {"authorization": auth_header}
response = await client.get(url, headers=headers)
```

Rationale:

- Ensures token context flows through service chain
- Allows downstream service to validate caller identity
- Enables audit trail

2. Error Response Consistency

Standard Error Response:

```
{
  "detail": "Human-readable error message"
}
```

With Additional Context:

```
{
  "detail": "Insufficient leave balance",
  "error_code": "INSUFFICIENT_BALANCE",
  "trace_id": "{correlation-id}",
  "service": "leave-service"
}
```

3. Asynchronous Event Publishing

Pattern:

```
try:
    # Update database
    db.commit()

    # Publish event (fire-and-forget)
    await publish_notification("leave.approved", {
        "leave_id": leave_id,
        "employee_username": username
    })
except Exception as e:
    logger.error(f"Failed to publish event: {e}")
    # Don't fail the main operation
```

Rationale:

- Don't block main operation for async notifications
- Log but don't propagate notification failures

4. Service-to-Service Call Resilience

Pattern:

```
async def call_external_service(url, timeout=5.0):
    try:
        async with httpx.AsyncClient(timeout=timeout) as client:
            response = await client.get(url)

            if response.status_code == 404:
                raise HTTPException(400, "Resource not found")
            elif response.status_code != 200:
                raise HTTPException(503, "Service unavailable")

            return response.json()

    except httpx.TimeoutException:
        raise HTTPException(503, "Service timeout")
    except httpx.RequestError as e:
        logger.error(f"Request failed: {e}")
        raise HTTPException(503, "Service unavailable")
```

Rationale:

- Consistent timeout handling
- Error propagation with appropriate status codes
- Logging for debugging

5. Database Transaction Handling

Pattern for Multi-Step Operations:

```

try:
    db.begin()

    # Step 1: Update leave request status
    leave_request.status = "APPROVED"
    db.add(leave_request)

    # Step 2: Deduct leave balance (might call external service)
    response = await deduct_balance(...)

    # Step 3: Update local state based on response
    # ...

    db.commit()

except SQLAlchemyError as e:
    db.rollback()
    raise HTTPException(500, "Database error")
except Exception as e:
    db.rollback()
    raise

```

Rationale:

- All-or-nothing updates
- Consistent data state
- Proper cleanup on failure

6. Service Discovery Best Practices

Pattern:

```

# Use service names instead of hardcoded IPs
EMPLOYEE_SERVICE_URL = os.getenv(
    "EMPLOYEE_SERVICE_URL",
    "http://employee-service:8002" # Default for compose
)

```

Benefits:

- Environment-agnostic
- Easy to reconfigure
- Docker Compose → Kubernetes migration friendly

7. Trace Context Correlation

Pattern:

```

# Extract trace ID from request headers
trace_id = request.headers.get("X-Trace-ID")

# Ensure it exists
if not trace_id:
    trace_id = str(uuid.uuid4())

# Include in all logs
logger.info(f"Leave approved", extra={"trace_id": trace_id})

# Forward to external services
headers = {"X-Trace-ID": trace_id}
await client.get(url, headers=headers)

```

Rationale:

- Correlate logs across services
- Trace end-to-end request flow
- Debug distributed issues

Deployment Considerations

Development Environment (Current)

```
docker-compose up -d
```

Summary

This Inter-Service Communication architecture provides:

- ✓ **Synchronous HTTP/REST** for immediate replies (check balance)
- ✓ **Asynchronous RabbitMQ** for event-driven notifications
- ✓ **Service Discovery with Consul** for dynamic location resolution
- ✓ **Comprehensive Error Handling** with proper HTTP status codes
- ✓ **Distributed Tracing** for observability
- ✓ **Token Propagation** for end-to-end authentication

Key Assumptions Summary

- Services communicate over Docker network with DNS resolution
- Single PostgreSQL database shared by all services
- JWT tokens valid for 24 hours with same secret key across services
- HTTP calls timeout at 5 seconds
- RabbitMQ provides persistent, guaranteed message delivery
- Consul runs continuously for service discovery
- Manager-employee hierarchy maintained in database

Next Steps

1. Add Circuit Breaker pattern for inter-service calls
2. Implement database-per-service for true service isolation
3. Set up Prometheus metrics collection
4. Configure centralized logging (ELK stack or similar)